UNIT V
Worked Case Study and Student Practice Activity

MongoDB, Database Operations and Cloud Deployment

Course Details	Information
Course Title	Full-Stack Web Development
Course Code	23CM4121
Unit	Unit V: MongoDB and Deploying Applications
Student Practice / Case Study	Student Management System (CRUD, Aggregation & Cloud Deployment)

PART A. Comprehensive Question

Case Study: Student Management System

An educational institution maintains student records, enrollment details, and academic grades in spreadsheets and manual files. The administration and faculty face significant challenges in tracking student progress, updating grades, querying course enrollments, and analyzing course-wise student metrics. The institution requires a modern, web-based Student Management System that securely stores student information in a centralized database and provides reliable access over the internet.

Design and develop a database-driven Student Management System using React, Node.js, Express.js, and MongoDB. Deploy the frontend and backend on suitable cloud platforms and make the application accessible through a public web address.

The application must support the following requirements:

- Store student details such as student ID (roll number), full name, institutional email, course/department, age, GPA, and enrollment date.
- Create and manage a MongoDB database and students collection using an appropriate document structure.
- Perform full CRUD operations: add a new student, display enrolled students, modify student details, and delete a student record.
- Write MongoDB shell queries to insert, retrieve, update, and delete documents.
- Use MongoDB aggregation pipelines to generate course-wise student counts and average GPA summaries.
- Create indexes (unique index on studentId/email, secondary index on course) to support efficient student lookup.
- Develop an Express.js REST API with comprehensive schema validation and robust error handling.
- Build an interactive React frontend with Vite to perform operations via API calls.
- Deploy the backend, frontend, and database using cloud platforms (MongoDB Atlas, Render, Vercel). Explain the role of hosting, domain names, and environment variables.
- Test the deployed application and confirm that data entered through the interface is stored in MongoDB and can be reliably retrieved.

PART B. Worked Solution

1. Analyse the requirements

Requirement	Solution
Student identity	A unique student ID (studentId) and unique institutional email identify each student.
Student details	name, email, course, age, gpa, and enrollmentDate are stored in each document.
Data storage	MongoDB database: student_db; collection: students.
Application operations	React sends HTTP requests to Express API routes; Express reads/writes MongoDB documents via Mongoose.
Reports	Aggregation calculates total student count and average GPA course-wise.
Performance	Unique indexes created on studentId and email; secondary index on course.
Deployment	MongoDB Atlas hosts the database; Render hosts the backend API; Vercel hosts the React frontend.

2. Design the MongoDB document

A student is represented as one BSON document in the students collection. A possible document is:

```
{
  "studentId": "STU-101",
  "name": "Alice Johnson",
  "email": "alice@univ.edu",
  "course": "Computer Science",
  "age": 20,
  "gpa": 3.85,
  "enrollmentDate": "2026-08-15T09:00:00.000Z"
}
```

Field design:

Field	Suggested data type	Purpose
studentId	String	Unique student code or roll number.
name	String	Student full legal name.
email	String	Unique institutional email address.
course	String	Academic program/department (e.g.

Field	Suggested data type	Purpose
		Computer Science).
age	Number	Age of the student (positive integer).
gpa	Number	Cumulative Grade Point Average (0.0 to 4.0 scale).
enrollmentDate	Date	Date of admission into the institution.

The MongoDB document also receives an automatically generated _id field (ObjectId). The application uses this identifier or the unique studentId when querying, updating, or deleting a student record.

3. Create the database and collection

In MongoDB Shell (mongosh), execute the following commands. A database is persisted when data is stored in it; creating a collection makes the intended structure explicit.

```
use student_db

db.createCollection("students")

db.students.insertMany([
  { studentId: "STU-101", name: "Alice Johnson", email: "alice@univ.edu", course: "Computer
Science", age: 20, gpa: 3.85 },
  { studentId: "STU-102", name: "Bob Smith", email: "bob@univ.edu", course: "Information
Technology", age: 22, gpa: 3.60 },
  { studentId: "STU-103", name: "Carol White", email: "carol@univ.edu", course: "Data Science",
age: 21, gpa: 3.75 }
])
```

Expected result: The students collection contains three sample student documents. The sample values are illustrative and can be expanded during application runtime.

4. Perform MongoDB CRUD operations

Operation	MongoDB query	Purpose
Create	db.students.insertOne({ studentId: "STU-104", name: "David Lee", email: "david@univ.edu", course: "Computer Science", age: 23, gpa: 3.90 })	Adds a student document.
Read	db.students.find({ course: "Computer Science" })	Retrieves students in Computer Science.
Update	db.students.updateOne({ studentId: "STU-101" }, { $set: { gpa: 3.95 } })	Modifies student GPA.

Operation	MongoDB query	Purpose
Delete	db.students.deleteOne({ studentId: "STU-104" })	Removes matching student document.

Useful additional queries:

```
// Display all students
db.students.find({})

// Find high-performing students with GPA >= 3.7
db.students.find({ gpa: { $gte: 3.7 } })

// Sort students by GPA, highest first
db.students.find({}).sort({ gpa: -1 })

// Count total registered students in the collection
db.students.countDocuments({})
```

5. Create indexes

An index allows MongoDB to locate documents quickly without scanning every document in the collection. Create unique indexes for studentId and email, and a secondary index for course-based queries.

```
db.students.createIndex({ studentId: 1 }, { unique: true })
db.students.createIndex({ email: 1 }, { unique: true })
db.students.createIndex({ course: 1 })

// View created indexes
db.students.getIndexes()
```

The unique studentId and email indexes guarantee that duplicate records cannot be inserted into the system. The course index speeds up filtering and aggregation queries. Indexes consume RAM and disk space, so they should align with frequent access patterns.

6. Use aggregation for course-wise student summary

The academic dean requires department-wise statistics on total student enrollment and average GPA. This is achieved using the MongoDB aggregation pipeline with $group and $sort stages.

```
db.students.aggregate([
  {
    $group: {
      _id: "$course",
      studentCount: { $sum: 1 },
      averageGpa: { $avg: "$gpa" }
    }
  },
  { $sort: { averageGpa: -1 } }
])
```

For the three sample records, the expected course totals are:

Course / Department	Student count	Average GPA
Computer Science	1	3.85
Data Science	1	3.75
Information Technology	1	3.60

7. Develop the Express.js backend

Create a folder named backend. Initialise the Node.js project and install the required packages:

```
npm init -y
npm install express mongoose cors dotenv
```

Create backend/package.json:

```
{
  "name": "student-backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "mongoose": "^8.8.4"
  }
}
```

Create backend/server.js:

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(express.json());

const allowedOrigins = (process.env.FRONTEND_ORIGIN || "http://localhost:5173")
  .split(",").map((origin) => origin.trim());
app.use(cors({ origin: allowedOrigins }));

const studentSchema = new mongoose.Schema({
  studentId: { type: String, required: true, unique: true, trim: true },
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
```

```
  course: { type: String, required: true, trim: true },
  age: { type: Number, required: true, min: 1 },
  gpa: { type: Number, required: true, min: 0.0, max: 4.0, default: 0.0 }
}, { timestamps: true });

studentSchema.index({ course: 1 });
const Student = mongoose.model("Student", studentSchema);

// Health endpoint
app.get("/api/health", (req, res) => {
  res.json({ status: "ok", message: "Student Management API is running" });
});

// Retrieve all students
app.get("/api/students", async (req, res) => {
  try {
    const students = await Student.find().sort({ name: 1 });
    res.json(students);
  } catch (err) {
    res.status(500).json({ message: "Unable to retrieve students" });
  }
});

// Create new student
app.post("/api/students", async (req, res) => {
  try {
    const student = await Student.create(req.body);
    res.status(201).json(student);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// Update student details
app.put("/api/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(
      req.params.id, req.body, { new: true, runValidators: true }
    );
    if (!student) return res.status(404).json({ message: "Student not found" });
    res.json(student);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// Delete student
app.delete("/api/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);
    if (!student) return res.status(404).json({ message: "Student not found" });
    res.json({ message: "Student deleted successfully" });
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// Aggregation summary route
```

```
app.get("/api/reports/course-summary", async (req, res) => {
  try {
    const summary = await Student.aggregate([
      {
        $group: {
          _id: "$course",
          studentCount: { $sum: 1 },
          averageGpa: { $avg: "$gpa" }
        }
      },
      { $sort: { averageGpa: -1 } }
    ]);
    res.json(summary);
  } catch (err) {
    res.status(500).json({ message: "Unable to generate summary" });
  }
});

const PORT = process.env.PORT || 5000;
async function start() {
  try {
    await mongoose.connect(process.env.MONGODB_URI, {
      dbName: process.env.DB_NAME || "student_db"
    });
    app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
  } catch (err) {
    console.error("Database connection failed:", err.message);
    process.exit(1);
  }
}
start();
```

Create backend/.env.example (use your own values in a private .env file):

```
MONGODB_URI=mongodb+srv://<username>:<password>@<cluster-url>/
DB_NAME=student_db
FRONTEND_ORIGIN=http://localhost:5173
NODE_ENV=development
```

API endpoint summary:

Method	Endpoint	Action
GET	/api/health	Check whether the backend server and DB are running.
GET	/api/students	Retrieve all students sorted by name.
POST	/api/students	Create and persist a new student document.
PUT	/api/students/:id	Update student details using

Method	Endpoint	Action
		MongoDB _id.
DELETE	/api/students/:id	Delete a student record using MongoDB _id.
GET	/api/reports/course-summary	Generate course-wise aggregation report.

8. Develop the React frontend

Create a Vite React application in a separate frontend folder:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev
```

Set the API base URL in frontend/.env:

```
VITE_API_URL=http://localhost:5000
```

Create frontend/src/App.jsx. This component displays the student list and provides add, edit, and delete operations through the Express API.

```
import { useEffect, useState } from "react";
import "./style.css";

const API = (import.meta.env.VITE_API_URL || "http://localhost:5000").replace(/\/$/, "") + "/api";
const emptyForm = { studentId: "", name: "", email: "", course: "", age: "", gpa: "" };

export default function App() {
  const [students, setStudents] = useState([]);
  const [form, setForm] = useState(emptyForm);
  const [editingId, setEditingId] = useState(null);
  const [message, setMessage] = useState("");

  async function loadStudents() {
    const response = await fetch(`${API}/students`);
    if (!response.ok) throw new Error("Could not load students");
    setStudents(await response.json());
  }

  useEffect(() => {
    loadStudents().catch((err) => setMessage(err.message));
  }, []);

  function handleChange(event) {
    setForm({ ...form, [event.target.name]: event.target.value });
  }
```

```
async function handleSubmit(event) {
  event.preventDefault();
  const payload = {
    ...form,
    age: Number(form.age),
    gpa: Number(form.gpa)
  };
  const url = editingId ? `${API}/students/${editingId}` : `${API}/students`;
  const response = await fetch(url, {
    method: editingId ? "PUT" : "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload)
  });
  const result = await response.json();
  if (!response.ok) { setMessage(result.message || "Save failed"); return; }
  setMessage(editingId ? "Student updated" : "Student added");
  setForm(emptyForm);
  setEditingId(null);
  await loadStudents();
}

function startEdit(student) {
  setEditingId(student._id);
  setForm({
    studentId: student.studentId,
    name: student.name,
    email: student.email,
    course: student.course,
    age: String(student.age),
    gpa: String(student.gpa)
  });
}

async function removeStudent(id) {
  if (!window.confirm("Delete this student record?")) return;
  const response = await fetch(`${API}/students/${id}`, { method: "DELETE" });
  const result = await response.json();
  if (!response.ok) { setMessage(result.message || "Delete failed"); return; }
  setMessage("Student deleted");
  await loadStudents();
}

return (
  <main className="container">
    <h1>Student Management System</h1>
    <p>Manage enrolled students and academic performance.</p>
    {message && <p role="status">{message}</p>}
    <form onSubmit={handleSubmit}>
      {Object.keys(emptyForm).map((key) => (
        <label key={key}>{key.toUpperCase()}
          <input
            name={key}
            value={form[key]}
            onChange={handleChange}
            type={key === "age" || key === "gpa" ? "number" : key === "email" ? "email" : "text"}
            min={key === "age" ? "1" : key === "gpa" ? "0" : undefined}
            max={key === "gpa" ? "4.0" : undefined}
            step={key === "gpa" ? "0.01" : "1"}
```

```
              required
            />
          </label>
        ))}
        <button type="submit">{editingId ? "Update Student" : "Add Student"}</button>
        {editingId && (
          <button type="button" onClick={() => { setEditingId(null); setForm(emptyForm); }}>
            Cancel Edit
          </button>
        )}
      </form>

      <h2>Enrolled Student Records</h2>
      <table>
        <thead>
          <tr><th>ID</th><th>Name</th><th>Email</th><th>Course</th><th>Age</th><th>GPA</
th><th>Actions</th></tr>
        </thead>
        <tbody>
          {students.map((s) => (
            <tr key={s._id}>
              <td>{s.studentId}</td><td>{s.name}</td><td>{s.email}</td>
              <td>{s.course}</td><td>{s.age}</td><td>{s.gpa}</td>
              <td>
                <button onClick={() => startEdit(s)}>Edit</button>
                <button onClick={() => removeStudent(s._id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </main>
  );
}
```

Create frontend/src/main.jsx:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode><App /></React.StrictMode>
);
```

Create frontend/src/style.css (a clean, responsive stylesheet):

```
body { font-family: Arial, sans-serif; margin: 0; background: #f4f6f8; }
.container { max-width: 1100px; margin: 30px auto; padding: 24px; background: white; border-radius:
10px; }
form { display: grid; grid-template-columns: repeat(3, 1fr); gap: 12px; margin-bottom: 24px; }
label { display: grid; gap: 5px; font-size: 13px; font-weight: bold; color: #334155; }
input { padding: 9px; border: 1px solid #bbb; border-radius: 5px; font-size: 14px; }
button { padding: 8px 14px; margin: 4px; cursor: pointer; border-radius: 5px; font-weight: bold; }
```

```
table { width: 100%; border-collapse: collapse; margin-top: 12px; }
th, td { border: 1px solid #ddd; padding: 9px; text-align: left; }
th { background: #eaf1f7; color: #1e3a8a; }
@media (max-width: 700px) { form { grid-template-columns: 1fr; } .container { margin: 10px;
padding: 12px; } }
```

9. Run and test the application locally

1. Start MongoDB Atlas or a local MongoDB instance and set MONGODB_URI in backend/.env.
2. From the backend folder, run npm install and then npm start. Confirm that the API starts after connecting to MongoDB.
3. From the frontend folder, run npm install and npm run dev. Open the local Vite URL shown in the terminal.
4. Add a student through the form. Confirm that it appears in the student table and in the MongoDB students collection.
5. Edit the student GPA and confirm that the updated value is displayed and stored.
6. Delete a test student and verify that it is removed from the collection.
7. Open /api/health and /api/reports/course-summary in the backend URL to check the health response and aggregation report.

Test case evaluation matrix:

Test case	Action	Expected result
TC1: Create	Submit a valid student form.	New student is stored in MongoDB and displayed in table.
TC2: Read	Open or reload the application.	Stored student documents are retrieved from MongoDB.
TC3: Update	Edit the GPA of a student record.	Updated GPA is persisted and visible after refresh.
TC4: Delete	Delete a test student.	Student is permanently removed from the list and database.
TC5: Validation	Submit an incomplete or invalid record (e.g. invalid email).	API rejects invalid payload and returns HTTP 400 error.
TC6: Aggregate	Request course summary endpoint.	Department-wise student counts and average GPA are returned.

Sample execution screens:

Use these examples to understand what evidence to capture. For a student submission, capture the real application, API response, and MongoDB collection from the student's own run.

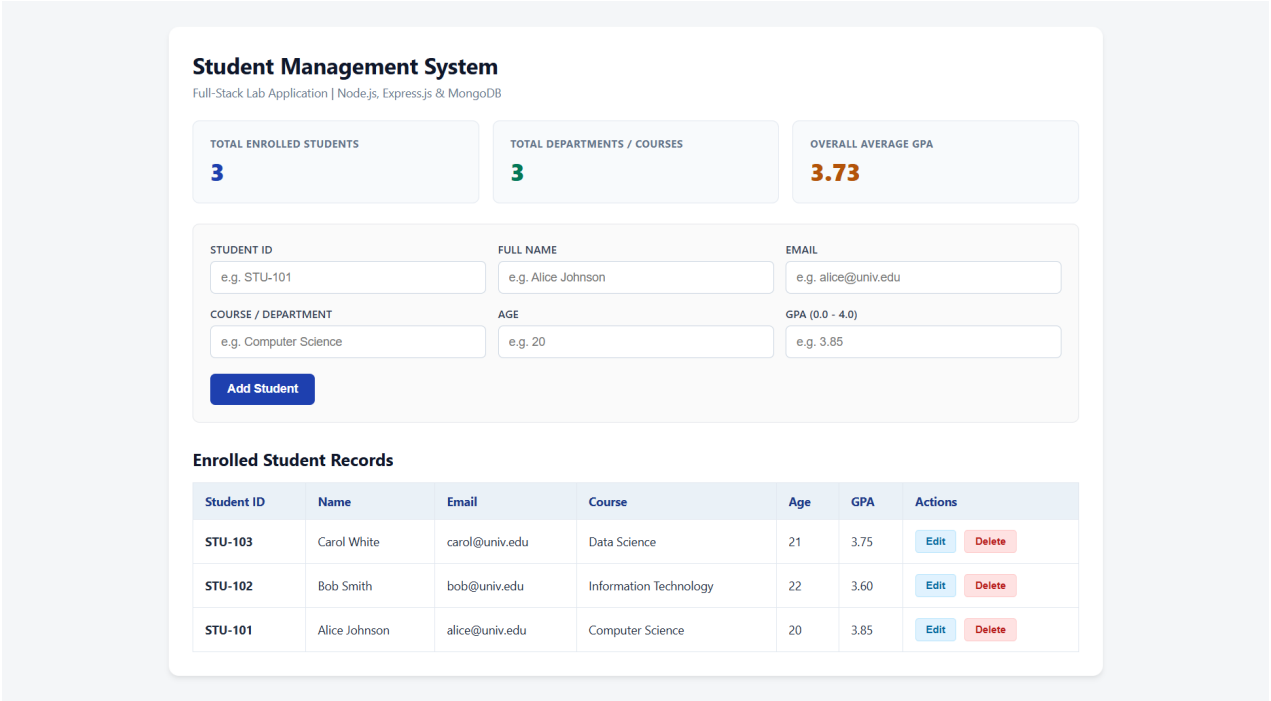


Figure 1. Student dashboard after update

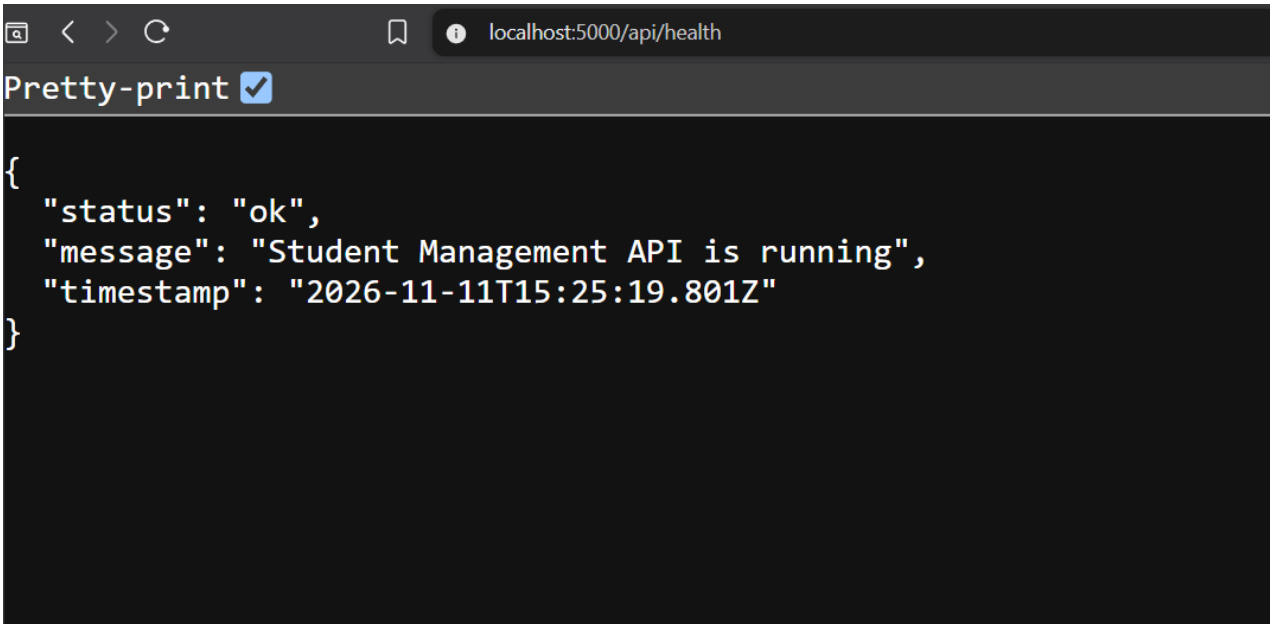


Figure 2. Backend health endpoint

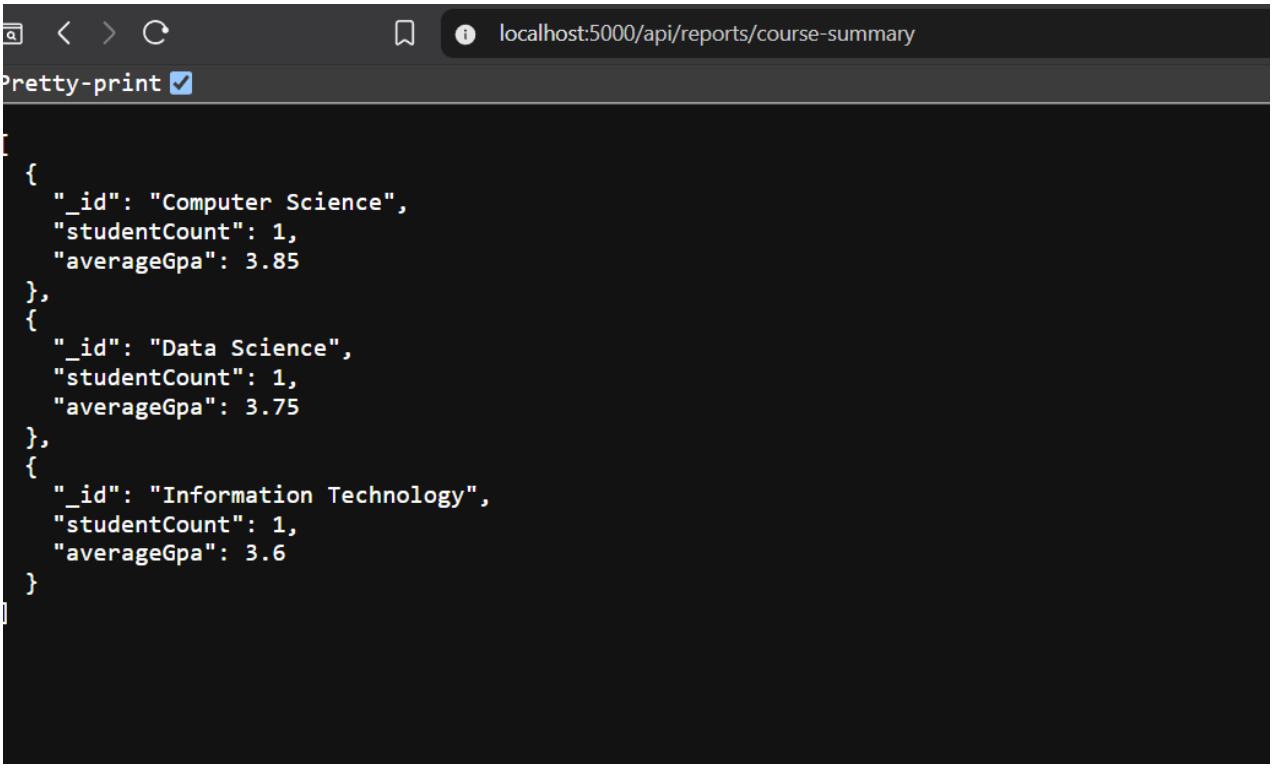


Figure 3. Course-wise aggregation response

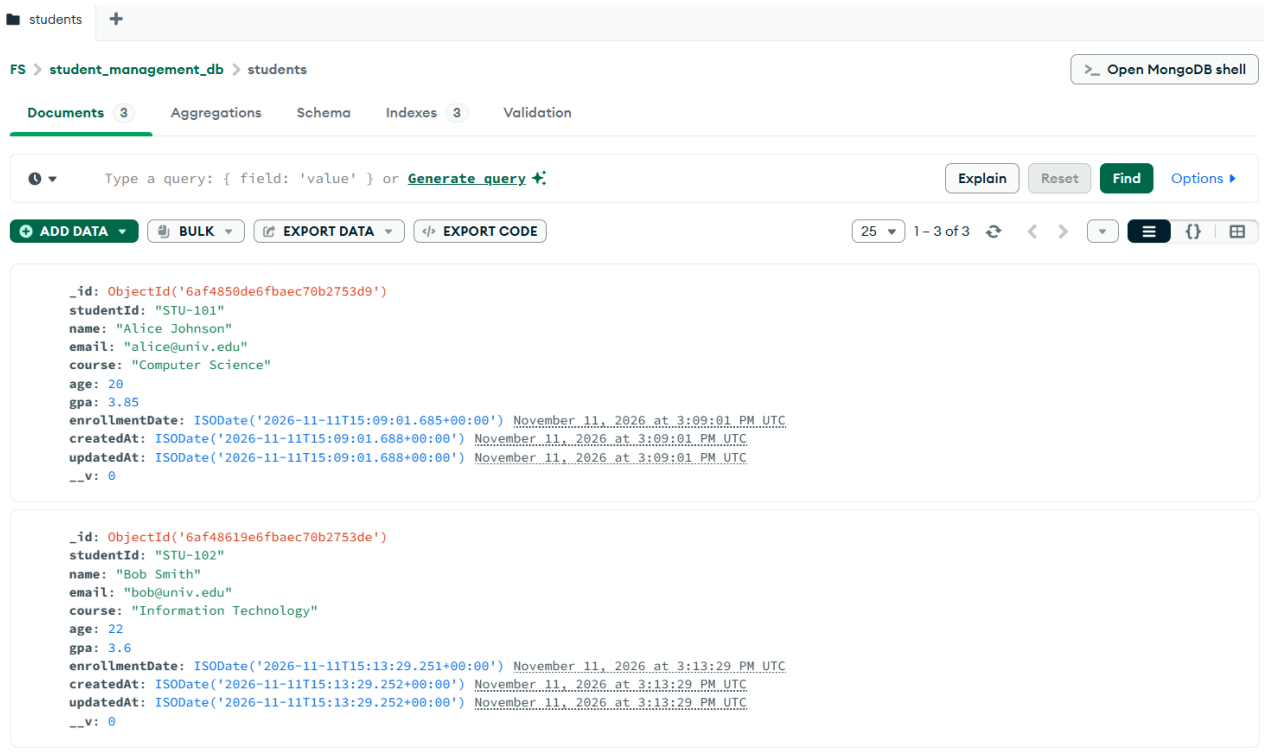


Figure 4. MongoDB document showing persisted student record



Figure 5. CRUD API test summary

Useful API test commands:

```
# Health check
curl http://localhost:5000/api/health

# Read all students
curl http://localhost:5000/api/students

# Aggregation summary
curl http://localhost:5000/api/reports/course-summary

# Create student
curl -X POST http://localhost:5000/api/students -H "Content-Type: application/json" \
  -d '{"studentId":"STU-104","name":"David Lee","email":"david@univ.edu","course":"Computer
Science","age":23,"gpa":3.90}'

# Update student (replace <id> with actual MongoDB _id or studentId)
curl -X PUT http://localhost:5000/api/students/<id> -H "Content-Type: application/json" \
```

```
  -d '{"gpa":3.98}'

# Delete student (replace <id> with actual MongoDB _id or studentId)
curl -X DELETE http://localhost:5000/api/students/<id>
```

10. Deploy the application using cloud platforms

Deployment architecture:

Layer	Platform	Responsibility
Frontend	Vercel	Hosts the built React SPA and provides a public HTTPS frontend URL.
Backend	Render	Runs the Node.js/Express REST API and provides a public HTTPS API URL.
Database	MongoDB Atlas	Fully managed cloud MongoDB cluster storing persistent student documents.

Deployment steps:

1. Create a MongoDB Atlas cluster. Create a database user, configure Network Access (whitelist 0.0.0.0/0 for cloud hosts or specific IPs), and copy the SRV connection string.

2. Push the backend and frontend source folders to a GitHub repository. Ensure .env files and node_modules are excluded via .gitignore.

3. On Render, create a Web Service linked to the backend repository. Set Build Command to 'npm install' and Start Command to 'npm start'.

4. Add backend environment variables in Render: MONGODB_URI, DB_NAME=student_db, and FRONTEND_ORIGIN=<your Vercel frontend URL>. Add NODE_ENV=production.

5. Deploy the backend and test its /api/health endpoint. Copy the public backend URL provided by Render.

6. On Vercel, import the frontend project repository. Set VITE_API_URL to the public Render backend base URL, then deploy the frontend.

7. Update FRONTEND_ORIGIN in Render with the deployed Vercel URL and restart the backend service. Test create, read, update, and delete operations from the public frontend URL.

8. For a custom institutional domain, register a domain and configure CNAME / A records in DNS as requested by Vercel/Render. Verify domain propagation.

Environment variable summary:

Variable	Where to set it	Example / value
MONGODB_URI	Render backend service	mongodb+srv:// admin:pass@cluster.mongodb.net/ (secret)

Variable	Where to set it	Example / value
DB_NAME	Render backend service	student_db
FRONTEND_ORIGIN	Render backend service	https://student-management.vercel.app
VITE_API_URL	Vercel frontend project	https://student-api.onrender.com

Security and deployment caution
Do not place MONGODB_URI in a Vite variable or frontend source code. Frontend variables are bundled into client-side JS and are visible to all users in the browser. Do not commit .env credentials to version control. Restrict MongoDB Atlas network access appropriately and use synthetic/dummy data for classroom exercises.

11. Expected final outcome

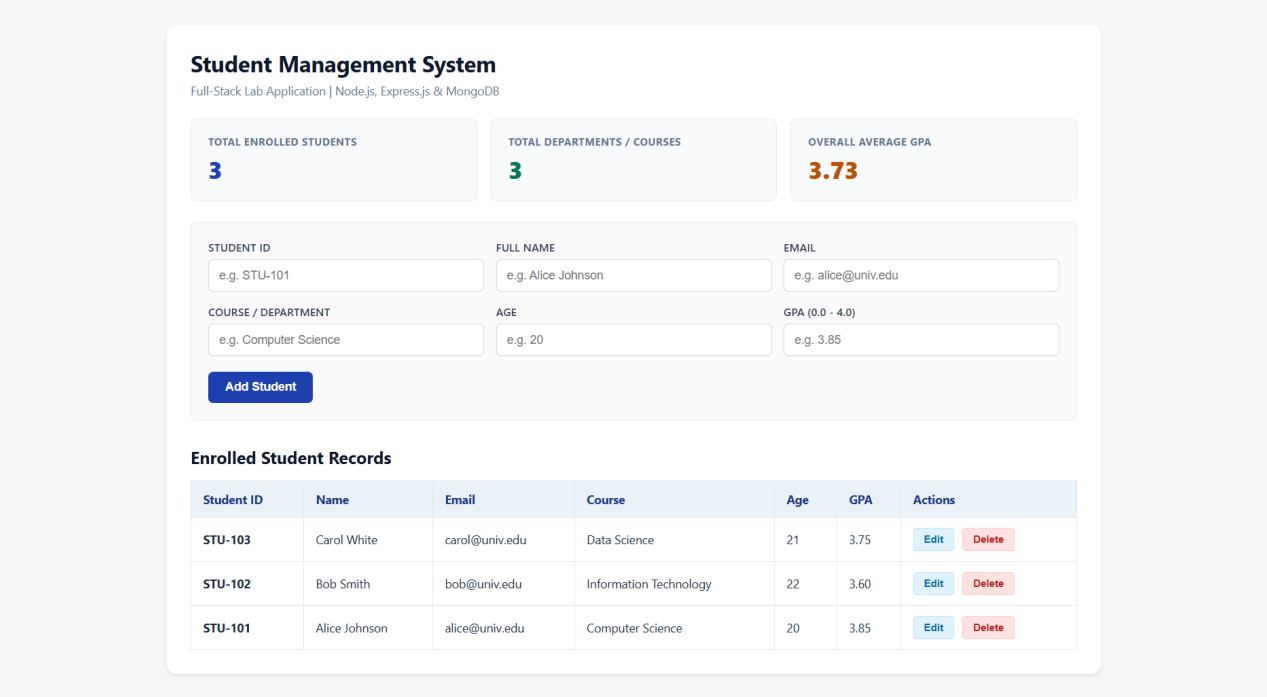
- A public frontend URL that opens the Student Management System application.
- A backend API URL that responds successfully to the health-check endpoint (/api/health).
- A MongoDB Atlas database containing persistent student documents in the students collection.
- Fully functional add, view, edit, and delete operations executable directly from the user interface.
- A course-wise aggregation report (/api/reports/course-summary) and configured database indexes.
- Verifiable evidence that the deployed frontend communicates with the cloud backend and database without CORS errors.

Students should capture their own screenshots during execution. The screenshots should show their actual running application, MongoDB documents, API test results, and cloud deployment dashboard. Do not submit illustrative or copied screenshots as execution evidence.

PART C. Output Screen Shots (Test Case Results):

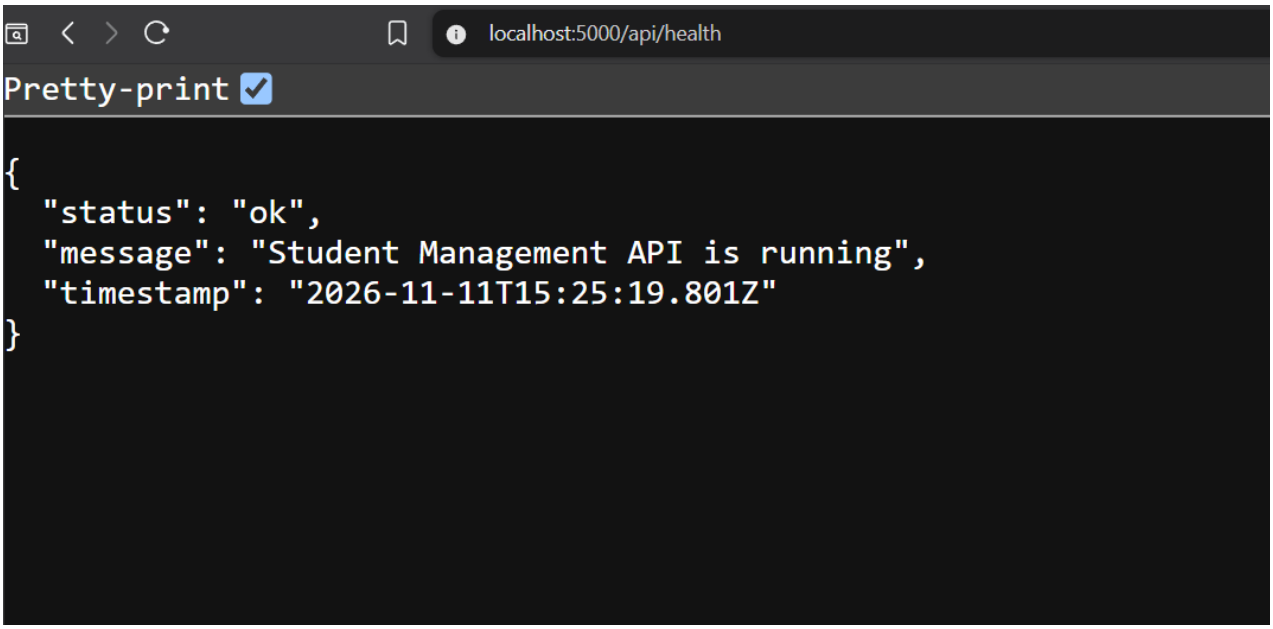
Screen 1: Student Management Dashboard

Shows the enrolled student list, summary count cards, average GPA, and edit/delete actions.



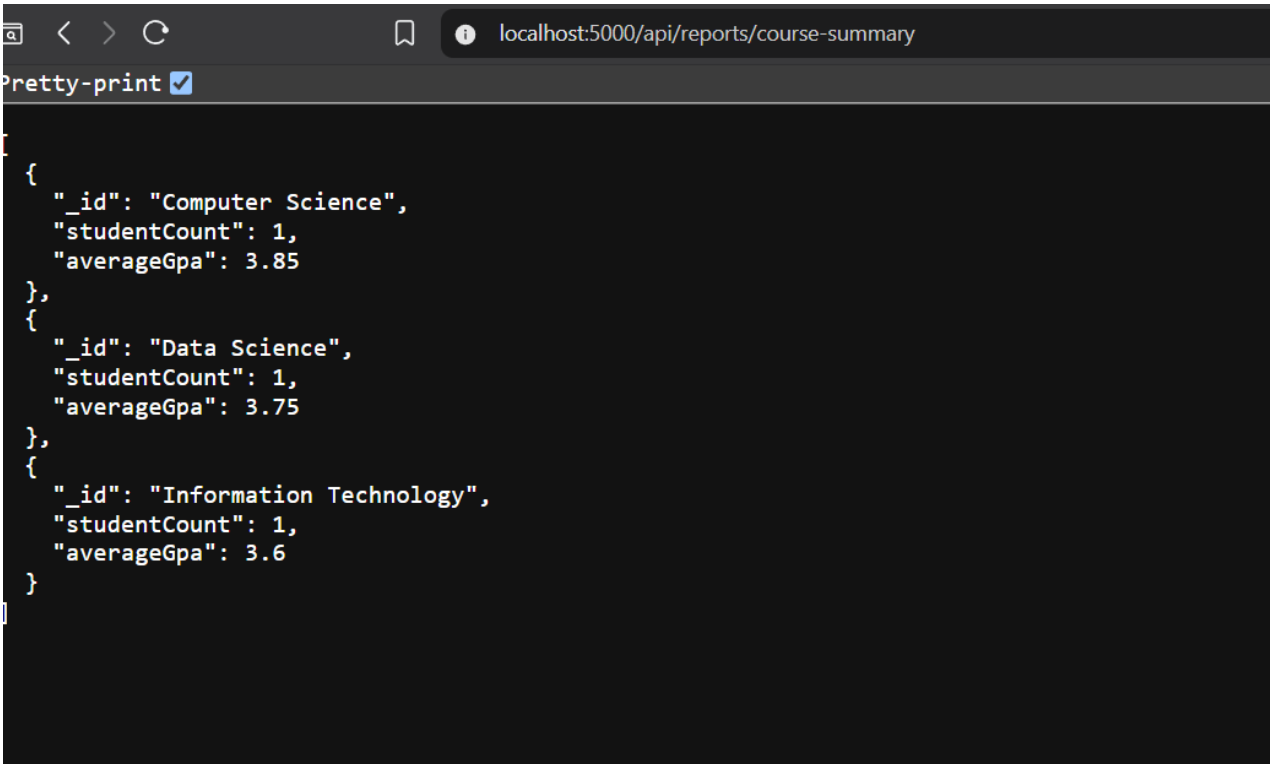
Screen 2: Backend API Health Check

Shows the successful HTTP 200 JSON response from /api/health.



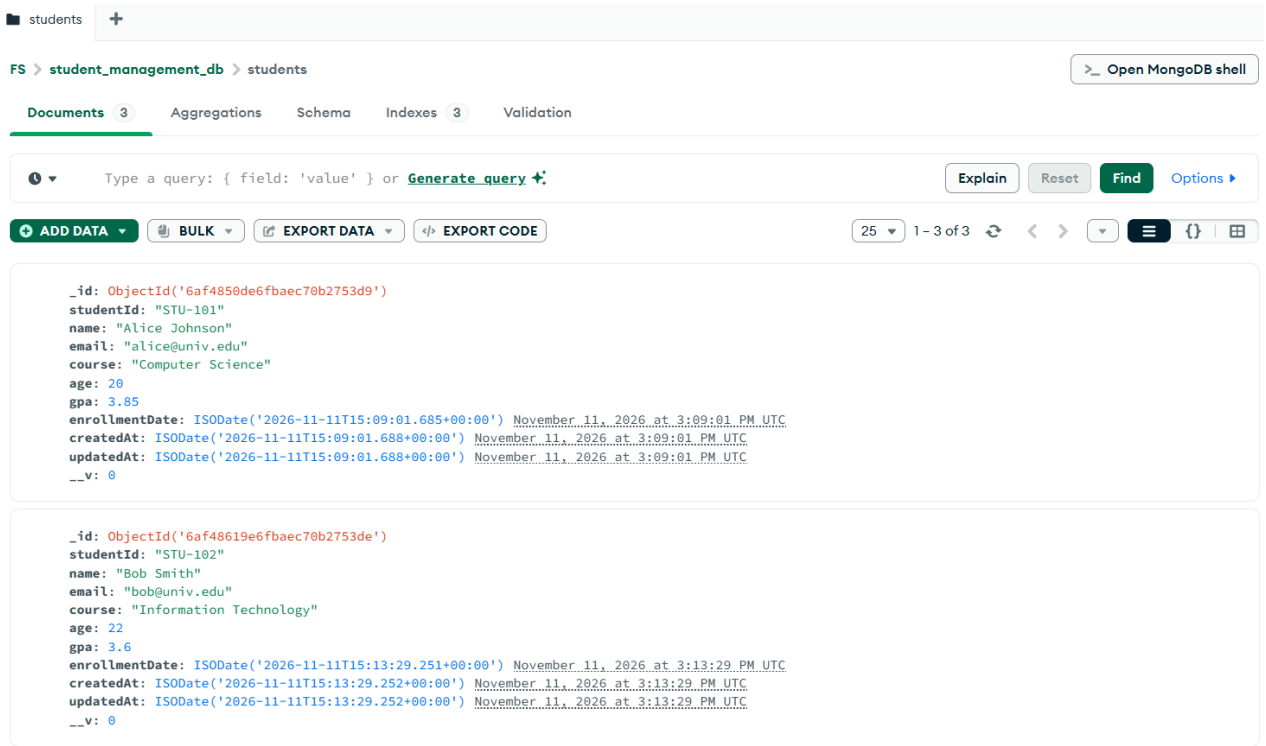
Screen 3: Course-wise Aggregation Report

Shows the MongoDB aggregation pipeline response with student count and average GPA by course.



Screen 4: MongoDB Atlas / Compass Document Collection

Shows the stored BSON student documents persisted in the cloud MongoDB database.



Screen 5: CRUD API Test Suite Execution

Shows the terminal cURL / Postman execution results across all CRUD endpoints.

```
PS C:\Users\Purna> irm http://localhost:5000/api/students | ConvertTo-Json -Depth 5
{
    "success":  true,
    "count":  3,
    "pagination":  {
                       "total":  3,
                       "page":  1,
                       "pages":  1,
                       "limit":  20
                   },
    "data":  [
                 {
                     "_id":  "6af48672e6fbaec70b2753e3",
                     "studentId":  "STU-103",
                     "name":  "Carol White",
                     "email":  "carol@univ.edu",
                     "course":  "Data Science",
                     "age":  21,
                     "gpa":  3.75,
                     "enrollmentDate":  "2026-11-11T15:14:58.719Z",
                     "createdAt":  "2026-11-11T15:14:58.720Z",
                     "updatedAt":  "2026-11-11T15:14:58.720Z",
                     "__v":  0
                 },
                 {
                     "_id":  "6af48619e6fbaec70b2753de",
                     "studentId":  "STU-102",
                     "name":  "Bob Smith",
                     "email":  "bob@univ.edu",
                     "course":  "Information Technology",
                     "age":  22,
                     "gpa":  3.6,
                     "enrollmentDate":  "2026-11-11T15:13:29.251Z",
                     "createdAt":  "2026-11-11T15:13:29.252Z",
                     "updatedAt":  "2026-11-11T15:13:29.252Z",
                     "__v":  0
                 },
                 {
                     "_id":  "6af4850de6fbaec70b2753d9",
                     "studentId":  "STU-101",
                     "name":  "Alice Johnson",
                     "email":  "alice@univ.edu",
                     "course":  "Computer Science",
                     "age":  20,
                     "gpa":  3.85,
                     "enrollmentDate":  "2026-11-11T15:09:01.685Z",
                     "createdAt":  "2026-11-11T15:09:01.688Z",
                     "updatedAt":  "2026-11-11T15:09:01.688Z",
                     "__v":  0
                 }
             ]
}
PS C:\Users\Purna>
```